



Lecture 2: Vector Databases Deep Dive

Understanding How Vector Search Actually Works

Part of the RAG Mastery Series



What You Will Learn Today

1. What are vectors and embeddings (deep understanding)
 2. How vector similarity search works
 3. The HNSW algorithm (the heart of vector search)
 4. Popular vector databases compared
 5. When to use which vector database
 6. Hands-on: Setting up ChromaDB and Qdrant
-

1. Vectors & Embeddings — The Deep Understanding

What is an Embedding?

An **embedding** is a way to represent text (or images, audio) as a list of numbers in a high-dimensional space, where **similar items are close together**.

```
"king"    → [0.8, 0.3, -0.2, 0.5, ...] (768 numbers)
"queen"   → [0.7, 0.4, -0.1, 0.6, ...] (close to king)
"apple"   → [-0.3, 0.9, 0.7, -0.4, ...] (far from king)
```

How Are Embeddings Created?

Embeddings are created by **neural network models** (embedding models). These models are trained on massive text data to understand semantic meaning.

Training Process (Simplified):

1. Take a large text corpus (Wikipedia, books, web pages)

2. Train a transformer model to predict masked words
3. The model learns to represent meaning in its hidden layers
4. We extract those hidden layer values as embeddings

Popular Embedding Models (2025-2026):

Model	Dimensions	Speed	Quality	Cost
text-embedding-3-small (OpenAI)	1536	Fast	Good	\$0.02/1M tokens
text-embedding-3-large (OpenAI)	3072	Medium	Excellent	\$0.13/1M tokens
all-MiniLM-L6-v2	384	Very Fast	Decent	Free
BGE-M3 (BAAI)	1024	Fast	Excellent	Free
E5-large-v2 (Microsoft)	1024	Medium	Very Good	Free
Jina Embeddings v3	1024	Fast	Excellent	Free (open)
Nomic Embed v2	768	Fast	Very Good	Free

Similarity Metrics

How do we measure "closeness" between vectors?

Cosine Similarity (Most Common):

```
cosine_sim(A, B) = (A · B) / (|A| × |B|)
```

Range: -1 to 1

1 = identical direction (very similar)

0 = perpendicular (unrelated)

-1 = opposite (very different)

Dot Product:

```
dot(A, B) = A[0]*B[0] + A[1]*B[1] + ... + A[n]*B[n]
```

Higher = more similar

Euclidean Distance:

```
dist(A, B) = sqrt((A[0]-B[0])2 + (A[1]-B[1])2 + ...)  
Lower = more similar
```

For RAG, cosine similarity is the standard.

2. Vector Search — The Core Problem

The Brute Force Approach

Given a query vector, compare it against EVERY vector in the database.

```
Database: 1,000,000 vectors  
Query: [0.12, -0.34, 0.56, ...]  
  
Compare query with vector_1 → similarity = 0.45  
Compare query with vector_2 → similarity = 0.89 ← good match!  
Compare query with vector_3 → similarity = 0.12  
...  
Compare query with vector_1000000 → similarity = 0.67  
  
Sort all by similarity → Return top-K
```

Problem: With 1 million vectors of 1536 dimensions each, this takes ~50ms. With 100 million vectors, it takes ~5 seconds. Too slow!

The Solution: Approximate Nearest Neighbor (ANN) Search

Instead of checking ALL vectors, use smart data structures to quickly narrow down candidates.

Key ANN Algorithms:

Algorithm	How It Works	Speed	Accuracy	Used By
HNSW	Graph-based navigation	Very Fast	High	Most VDBs
IVF	Clustering + search nearby	Fast	Medium	FAISS, Milvus
PQ	Compression + approximation	Very Fast	Lower	FAISS

ScaNN	Google's optimized ANN	Very Fast	High	Google
DiskANN	SSD-optimized graph search	Fast	High	Microsoft

3. HNSW — The Algorithm Behind Most Vector DBs

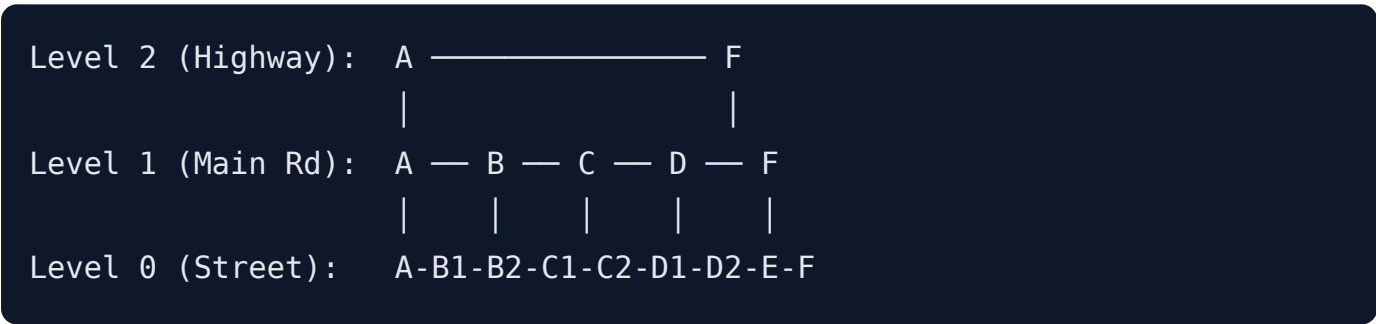
HNSW = Hierarchical Navigable Small World

This is THE algorithm you need to understand. It's used by:

- Qdrant, Weaviate, Pinecone, ChromaDB, pgvector, and most others

How HNSW Works (Simple Analogy)

Imagine a city with multiple levels of roads:



Search Process:

1. Start at the top level (highway) — few nodes, fast navigation
2. Find the closest node at this level
3. Go down one level — more nodes, more detail
4. Repeat until you reach the bottom level
5. Return the closest nodes found

Why it's fast: Instead of checking all N vectors, you only check $O(\log N)$ vectors.

1,000 vectors → ~10 comparisons (instead of 1000)

1,000,000 vectors → ~20 comparisons (instead of 1,000,000)

100M vectors → ~27 comparisons (instead of 100,000,000)

HNSW Parameters

Parameter	What It Does	Default	Trade-off
M	Max connections per node	16	Higher = better recall, more memory
ef_construction	Search depth during build	200	Higher = better index, slower build
ef_search	Search depth during query	100	Higher = better recall, slower search

4. Popular Vector Databases — Detailed Comparison

ChromaDB

```
# Easiest to start with
import chromadb
client = chromadb.Client()
collection = client.create_collection("my_docs")
collection.add(
    documents=["chunk 1 text", "chunk 2 text"],
    ids=["id1", "id2"]
)
results = collection.query(query_texts=["my question"], n_results=3)
```

-  Easiest setup, great for prototyping
-  Python-native, good LangChain integration
-  Not for production at scale
-  Limited filtering capabilities

Qdrant

```
# Production-grade, Rust-based
from qdrant_client import QdrantClient
client = QdrantClient("localhost", port=6333)
client.create_collection(
    collection_name="my_docs",
    vectors_config=VectorParams(size=384, distance=Cosine)
)
client.upsert(
    collection_name="my_docs",
    points=[
        PointStruct(id=1, vector=[0.1, 0.2, ...], payload={"text":
"chunk 1"}),
    ]
)
```

-  Very fast (written in Rust)
-  Rich filtering (filter by metadata while searching)
-  Open source + cloud option
-  Good for production
-  Need to run a server

Pinecone

```
# Fully managed, no server to run
import pinecone
pinecone.init(api_key="your-key")
index = pinecone.Index("my-docs")
index.upsert(vectors=[("id1", [0.1, 0.2, ...], {"text": "chunk"})])
results = index.query(vector=[0.1, 0.2, ...], top_k=3)
```

-  Zero infrastructure management
-  Auto-scaling
-  Great for startups
-  Can get expensive at scale
-  Vendor lock-in

Weaviate

```
# GraphQL API, hybrid search built-in
import weaviate
client = weaviate.Client("http://localhost:8080")
client.data_object.create(
    class_name="Document",
    data_object={"text": "chunk 1"},
    vector=[0.1, 0.2, ...]
)
```

-  Built-in hybrid search (BM25 + vector)
-  GraphQL API
-  Multi-modal support
-  Heavier to run

pgvector

```
# Add vectors to your existing PostgreSQL
CREATE EXTENSION vector;
CREATE TABLE documents (
    id SERIAL PRIMARY KEY,
    content TEXT,
    embedding vector(384)
);
SELECT * FROM documents
ORDER BY embedding <=> '[0.1, 0.2, ...]':vector
LIMIT 5;
```

-  Use your existing PostgreSQL
-  ACID transactions
-  No extra infrastructure
-  Not as fast as dedicated VDBs
-  Limited to single node

FAISS

```
# Facebook's library (not a database!)
import faiss
import numpy as np

dimension = 384
index = faiss.IndexHNSWFlat(dimension, 16) # HNSW index
index.add(np.array(embeddings).astype('float32'))
distances, indices = index.search(np.array([query]).astype('float32'),
k=5)
```

-  Extremely fast
-  Battle-tested at Facebook scale
-  GPU support
-  Not a database (no persistence, no metadata)
-  No real-time updates easily

Decision Guide: Which Vector DB to Use?

```
START
|
|— Just learning / prototyping? → ChromaDB
|
|— Production, small-medium scale? → Qdrant
|
|— Don't want to manage servers? → Pinecone
|
|— Already using PostgreSQL? → pgvector
|
|— Need hybrid search built-in? → Weaviate
|
|— Maximum speed, in-memory? → FAISS
|
|— Enterprise, billions of vectors? → Milvus
```

5. Vector Index Types in Detail

Flat Index (Brute Force)

- Checks every vector
- 100% accurate
- Very slow for large datasets
- Use only for small datasets (< 10K vectors)

IVF (Inverted File Index)

1. Cluster all vectors into groups (using K-means)
2. At query time, only search the nearest clusters
3. Much faster, slight accuracy loss

```
Clusters: [A, B, C, D, E, F, G, H]  
Query is closest to clusters B and D  
Only search vectors in B and D → much faster!
```

PQ (Product Quantization)

- Compresses vectors to save memory
- Splits vector into sub-vectors, quantizes each
- Enables searching billions of vectors in RAM
- Trade-off: some accuracy loss

HNSW + IVF + PQ (Combined)

Production systems often combine:

- IVF to narrow down clusters
- HNSW for navigation within clusters
- PQ for memory compression

6. Metadata Filtering — A Critical Feature

In real RAG systems, you often need to filter by metadata:

```
# "Find documents about refunds, but only from 2024, only from the
finance department"

# Qdrant filtering
results = client.query_points(
    collection_name="docs",
    query=query_vector,
    query_filter=Filter(
        must=[
            FieldKey("year").match(2024),
            FieldKey("department").match("finance"),
        ]
    ),
    limit=5
)
```

Why this matters: Without filtering, you might retrieve relevant chunks from the wrong department or wrong year. Metadata filtering is essential for enterprise RAG.

7. Quick Summary

Embeddings: Text → Numbers (vectors) that capture meaning
Similarity: Cosine similarity measures how close vectors are
ANN Search: Find similar vectors without checking all of them
HNSW: Graph-based navigation, the standard algorithm
Vector DBs: ChromaDB (easy), Qdrant (fast), Pinecone (managed)
Filtering: Always filter by metadata in production systems



Homework

1. Install ChromaDB and create a collection with 20 text chunks
2. Try different embedding models and compare search quality
3. Install Qdrant (Docker) and compare speed with ChromaDB
4. Implement metadata filtering (filter by category, date)

Next Lecture: Vectorless RAG & PageIndex — The new paradigm that challenges vector databases